

NxN CNN Convolution Accelerator for Edge-AI Vision

IEEE SSCS Egypt Chapter – 2026 Student Design Competition

Design Report

Team: Bit Streamers
Ain Shams University, Cairo, Egypt

September 2026

GitHub Repository:

<https://github.com/MohamedAtta22/CNN-convolution-accelerator->

Contents

1	Overview	4
1.1	Objective	4
1.2	Design Philosophy	4
1.3	High-Level Architecture	4
1.4	Specification Compliance Summary	5
2	Component: window_generator	6
2.1	Purpose	6
2.2	Interface	6
2.3	Algorithm	6
2.3.1	Worked Example	7
2.4	Parameterization and Generality	7
2.5	Resource Cost	7
3	Component: line_buffer	7
3.1	Purpose	7
3.2	Design	8
3.3	Sizing and the KERNEL.SIZE=1 Edge Case	8
3.4	Interaction with window_generator	8
4	Component: kernel_memory	8
4.1	Purpose	8
4.2	Interface	9
4.3	Design	9
4.4	Why Runtime-Loadable Coefficients, Fixed-at-Synthesis Kernel Size	9
4.5	Interaction with the Control FSM	9
5	Component: mac_array	10
5.1	Purpose	10
5.2	Interface	10
5.3	Design	10

5.4	Resource Cost and the DSP Question	10
5.5	Generality	10
6	Component: <code>adder_tree</code>	11
6.1	Purpose	11
6.2	Interface	11
6.3	Why a Generic Tree, Not a Hand-Built One	11
6.4	Algorithm	11
6.5	A Portability Lesson: <code>generate if</code> versus a Runtime Ternary	12
6.6	Latency Tradeoff Versus the Original Hand-Built Tree	12
6.7	Resource Cost	12
7	Component: <code>post_process</code>	12
7.1	Purpose	13
7.2	Interface	13
7.3	Logic and Priority Order	13
7.4	Why Saturation, Not Truncation or Rounding	13
7.5	Verified Boundary Behavior	14
8	Component: <code>output_fifo</code>	14
8.1	Purpose	14
8.2	Packing Scheme	14
8.3	Depth and Prefill	14
8.4	A Real Bug: the Hardcoded Prefill Threshold	14
8.5	Interface	15
9	Top Level: <code>cnn_accelerator</code> and its Control FSM	15
9.1	Purpose	15
9.2	Control FSM	15
9.3	Aligning Output Metadata with Pipeline Latency	16
9.4	Multi-Kernel Bank Replication	16
9.5	Parameter Summary	17
9.6	Resource Cost	17
10	Fixed-Point / Precision Analysis	17
10.1	Input Precision	17
10.2	Kernel Precision	17
10.3	Bit Growth Through the Pipeline	17
10.4	Why 16 Bits Is Enough for the Per-Tap Product	18
10.5	Why 32 Bits Is Enough for the Accumulator, for Any Realistic N	18
10.6	Output Precision and Saturation	18
11	Verification Methodology	18
11.1	Approach	18
11.2	Self-Checking SystemVerilog Testbench: <code>cnn_accelerator_tb.sv</code>	19
11.3	Independent Golden-Model Testbench: <code>golden_model.py + cnn_accelerator_golden_tb.sv</code>	20
11.4	Directed Saturation-Boundary Testbench: <code>cnn_accelerator_saturation_tb.sv</code>	21
11.5	Directed ReLU Testbench: <code>cnn_accelerator_relu_tb.sv</code>	22

11.6 Results Summary	23
11.7 Bugs Found During Verification	23
12 FPGA Synthesis and Implementation Results	24
12.1 Target and Flow	24
12.2 Utilization	24
12.3 Timing: Investigating an Apparent Failure	24
12.4 Power	25
13 Figure of Merit and Design Tradeoffs	25
13.1 Computation	25
13.2 Why DSPs Were Not Forced	26
13.3 Why Maximum Frequency Was Not Pushed	26
13.4 Multi-Kernel Scaling: An Economies-of-Scale Argument	26
14 Conclusion	27

1 Overview

1.1 Objective

This project implements a parameterizable, pipelined hardware accelerator for 2D convolution, targeting the IEEE SSCS Egypt Chapter 2026 Student Design Competition’s Edge-AI vision brief. The accelerator streams a grayscale image (or single-channel feature map) one pixel per cycle, convolves it against one or more $N \times N$ kernels with stride 1, and produces a stream of 16-bit signed results with optional ReLU activation, meeting every mandatory specification and three of the four optional bonus features.

1.2 Design Philosophy

Three decisions shaped every module described in this report:

1. **Parameterize everything that can vary, not just what the spec requires.** `KERNEL_SIZE`, `NUM_KERNELS`, `IMAGE_WIDTH`, and `IMAGE_HEIGHT` are all synthesis-time parameters. The competition only requires that N be configurable at synthesis time, this design additionally makes the number of parallel kernel banks and the target image dimensions first-class parameters, so the same RTL scales from a minimal $3 \times 3 / 32 \times 32$ configuration up to arbitrary kernel sizes and multi-channel configurations without touching module internals.
2. **Every pipeline stage that reduces data (the adder tree) is written generically, not hand-unrolled for one kernel size.** A hand-built 9-input reduction tree is easy to write for a fixed 3×3 kernel; it does not generalize. Section 6 describes the generic, self-sizing reduction tree used instead, along with a real portability issue it surfaced during verification.
3. **Verify independently, not just thoroughly.** Beyond a conventional self-checking testbench, this design is checked against a from-scratch Python/NumPy reference model with no shared code path with the RTL (satisfying the competition’s cross-language verification requirement in the strictest sense), plus two additional directed testbenches purpose-built to close specific coverage gaps identified by inspecting what the first two testbenches actually exercised, rather than assuming coverage. Section 11 covers this in depth, including two real bugs this process found.

1.3 High-Level Architecture

Figure 1 shows the top-level structure. A pixel stream feeds a single shared sliding-window generator; `NUM_KERNELS` independent coefficient banks each drive their own multiply-accumulate-and-saturate chain against that one shared window, so that all banks produce a result for a given window position *on the same clock cycle*. This is a deliberate architectural choice, not an implementation detail: the competition draws a distinction between simply supporting multiple kernels (which could be done by re-running the image once per kernel) and the stronger, optional “multi-channel-in-one-cycle” bonus, where multiple kernel results are produced simultaneously. This design targets the stronger interpretation directly, at the cost of replicating the multiply-accumulate hardware `NUM_KERNELS` times, a tradeoff discussed quantitatively in Section 13.

Sections 2 through 8 describe each module in the datapath in detail, in the order data flows through them. Section 9 then describes how the top-level module wires these together and sequences an image pass with a control FSM. Section 10 treats the fixed-point design as its own topic, since it

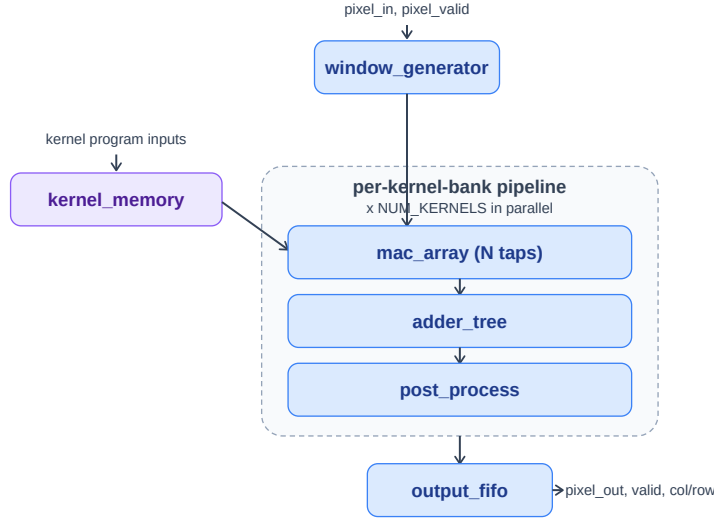


Figure 1: Top-level architecture. `window_generator` and `kernel_memory` feed a per-kernel-bank pipeline (`mac_array` → `adder_tree` → `post_process`) replicated `NUM_KERNELS` times in parallel; all banks write into a single, correspondingly widened `output_fifo`.

cuts across every module. Sections 11 through 13 cover verification, measured FPGA results, and the required Figure of Merit analysis.

1.4 Specification Compliance Summary

#	Requirement	How this design meets it
1	Minimum 32×32 input	<code>IMAGE_WIDTH/IMAGE_HEIGHT</code> are synthesis-time parameters, default 32×32; larger sizes require no RTL changes.
2	Unsigned fixed-point input	8-bit unsigned <code>pixel_in</code> (Q8.0), matching the standard 0–255 grayscale range (Section 10).
3	$N \times N$ programmable kernel	<code>KERNEL_SIZE</code> parameter (default 3); coefficients are runtime-loaded via <code>kernel_we/kernel_sel/kernel_addr/kernel_data</code> (Section 4).
4	8-bit signed kernel coefficients	<code>kernel_data</code> is signed [7:0], stored per-tap in <code>kernel_memory</code> .
5	Stride 1	Fixed by construction: <code>window_generator</code> advances the window by exactly one column per valid pixel (Section 2).
6	Minimum 16-bit signed output, overflow handling	32-bit accumulator saturated to signed 16-bit in <code>post_process</code> (Section 7); exact boundary behavior verified directly (Section 11).
7	ReLU (bonus)	Optional, per-cycle <code>relu_enable</code> input; its precedence over saturation is verified directly, not assumed.

#	Requirement	How this design meets it
8	Golden-model verification	Independent Python/NumPy reference model with zero shared code with the RTL (Section 11).
9	FPGA synthesis/implementation results	Vivado, xc7z020clg400-1 (PYNQ-Z2); full place-and-route, not synthesis-only estimates (Section 12).
–	Pipelined, one output/cycle (bonus)	Verified directly via back-to-back-cycle checks in two independent testbenches, not assumed from the architecture.
–	Multiple kernels (bonus)	NUM_KERNELS parallel banks share one window and produce results the same cycle – the stronger multi-channel-in-one-cycle tier.
–	Board demonstration (bonus)	not attempted.

2 Component: window_generator

2.1 Purpose

`window_generator` converts a raster-order pixel stream into a sequence of $N \times N$ sliding windows, one per valid output position, at a sustained rate of one window per cycle once enough of the image has been seen. It is the only module in the design that tracks position within the image; every downstream module operates purely on whatever window it is handed, with no notion of row or column.

2.2 Interface

Port	Dir.	Description
<code>pixel_valid</code>	in	Asserted for one cycle per incoming pixel.
<code>pixel_in</code>	in	8-bit unsigned pixel value, valid when <code>pixel_valid</code> is high.
<code>window</code>	out	<code>KERNEL_SIZE</code> × <code>KERNEL_SIZE</code> array of 8-bit pixels, the current window, valid when <code>window_valid</code> is high.
<code>window_valid</code>	out	One-cycle pulse: <code>window</code> holds a complete, in-bounds window this cycle.
<code>window_col</code> , <code>window_row</code>	out	Output-space coordinates of the window's top-left corner, valid alongside <code>window_valid</code> .

2.3 Algorithm

The module maintains two free-running counters, `current_col` and `current_row`, that track the raster position of the most recently arrived pixel. Building the actual window content is the more interesting part, and is split between this module and its `line_buffer` submodule (Section 3):

- `line_buffer` holds the `KERNEL_SIZE`–1 complete rows above the current one, and exposes them as `prev_row[0..KERNEL_SIZE-2]`, `prev_row[0]` being the most recently completed row.
- For each of the `KERNEL_SIZE` row-lanes of the window, a small shift register holds the `KERNEL_SIZE`–1 most recently seen columns for that lane. Each cycle, the lane's newest value (the current

pixel for the bottom lane, or the corresponding `prev_row` entry for every lane above it) shifts in, and the window for this cycle is assembled from these shift registers plus that newest value.

A window is only declared valid once both `current_row` $\geq$ `KERNEL_SIZE`–1 and `current_col` $\geq$ `KERNEL_SIZE`–1, i.e., once enough rows and columns have streamed by to fill every lane’s shift register with real image data rather than reset values. `window_col/window_row` are then simply `current_col/current_row` minus `KERNEL_SIZE`–1, translating from “pixels seen so far” coordinates into “output position” coordinates.

2.3.1 Worked Example

For `KERNEL_SIZE`=3 on an image at least 3 columns wide: the first window becomes valid the instant the pixel at physical position (row 2, col 2) arrives, the third pixel of the third row. At that point, the shift registers for lanes 0 and 1 (fed from `prev_row[1]` and `prev_row[0]`, i.e., rows 0 and 1) hold columns 0–1 of their respective rows, lane 2’s shift register holds columns 0–1 of the current row, and the incoming pixel supplies column 2 of the current row, together forming the complete 3×3 window covering rows 0–2, columns 0–2. `window_col/window_row` both read 0, correctly identifying this as output position (0,0). Every subsequent cycle (while still streaming pixels) advances the window by exactly one column, satisfying the required stride of 1, until the end of a row is reached and the window resets to column 0 of the next output row.

2.4 Parameterization and Generality

The row-lane/shift-register structure above generalizes cleanly to any `KERNEL_SIZE`: an $N \times N$ window needs N row-lanes, each needing $N - 1$ shift-register taps (the N -th column of each lane always comes directly from the newest incoming value for that lane). This was verified by hand for the degenerate `KERNEL_SIZE`=1 case, zero shift-register taps, and the “window” reduces correctly to just the incoming pixel itself, as well as confirmed to reduce to the original, hand-built 3×3 -only version of this module when `KERNEL_SIZE`=3.

2.5 Resource Cost

In the default $32 \times 32 / 3 \times 3$ /single-kernel configuration, `window_generator` (including its `line_buffer` instance) synthesizes to 376 LUTs, independent of `NUM_KERNELS`, since exactly one instance of this module is shared across every kernel bank (Section 13 returns to why this matters for the Figure of Merit as `NUM_KERNELS` scales).

3 Component: line_buffer

3.1 Purpose

A sliding-window convolution needs access to several complete prior rows of the image simultaneously, but the accelerator only ever receives one pixel per cycle. `line_buffer` is the memory that makes the previous `KERNEL_SIZE`–1 rows available on demand, so `window_generator` never needs to re-read pixels that have already streamed past.

3.2 Design

The module holds `KERNEL_SIZE-1` independent row-length delay lines (each `IMAGE_WIDTH` pixels deep), addressed by the current column. Every cycle a pixel arrives, each delay line’s contents are cascaded down by one position before the newest pixel is written into line 0:

Listing 1: Cascaded shift-and-write, from `line_buffer.sv`

```

1 for (int i = NUM_LINES - 1; i > 0; i--)
2     buffer[i][column] <= buffer[i-1][column];
3 buffer[0][column] <= pixel_in;
```

The descending loop order matters: because SystemVerilog non-blocking assignments (`<=`) capture their right-hand-side value at the start of the clock edge rather than after each statement executes, every line reads the value its neighbor held *before* this edge, not the value just written this same edge. This is what gives each line exactly a one-row delay relative to the line below it, and is the reason the loop counts down rather than up, ascending order would still be logically correct under non-blocking semantics, but the descending form makes the intended data-cascade direction visually explicit.

`prev_row[0]` (line 0) therefore always holds the most recently *completed* row, and `prev_row[KERNEL_SIZE-2]` holds the oldest row still needed by the window.

3.3 Sizing and the `KERNEL_SIZE=1` Edge Case

Total storage is $(\text{KERNEL_SIZE}-1) \times \text{IMAGE_WIDTH} \times 8$ bits. For the default 32-wide, 3×3 configuration this is $2 \text{ rows} \times 32 \text{ pixels} \times 8 \text{ bits} = 512$ bits, small enough that Vivado maps it to distributed (LUT-based) RAM rather than a dedicated Block RAM, which is reflected in the measured 0 BRAM usage reported in Section 12. For the degenerate case `KERNEL_SIZE=1`, no row history is needed at all; the module’s generate block produces a trivial pass-through with zero storage rather than a zero-depth (and therefore illegal) memory array.

3.4 Interaction with `window_generator`

`line_buffer` is purely a storage element, it has no notion of validity or window boundaries. All boundary logic (when a window is valid, what coordinates it corresponds to) lives in `window_generator`, described in Section 2. This separation keeps `line_buffer` reusable as a plain delay-line primitive independent of how its output is consumed.

4 Component: `kernel_memory`

4.1 Purpose

`kernel_memory` stores the coefficients for every kernel bank and exposes a simple write interface for loading them at runtime, satisfying the requirement that kernel coefficients be “programmable or configurable,” as distinct from `KERNEL_SIZE` itself, which is fixed at synthesis time (Section 10).

4.2 Interface

Port	Dir.	Description
<code>kernel_we</code>	in	Write enable.
<code>kernel_sel</code>	in	Selects which of the <code>NUM_KERNELS</code> banks this write targets.
<code>kernel_addr</code>	in	Tap index within the selected bank (0 to $\text{KERNEL_SIZE}^2 - 1$, row-major).
<code>kernel_data</code>	in	Signed 8-bit coefficient value.
<code>kernel</code>	out	Full $\text{NUM_KERNELS} \times \text{KERNEL_SIZE}^2$ array of stored coefficients.

4.3 Design

Storage is a two-dimensional register array, `kernel[bank][tap]`. A write lands only when both `kernel_sel` and `kernel_addr` are in range:

Listing 2: Guarded write, from `kernel_memory.sv`

```

1 if (kernel_we && (kernel_sel < NUM_KERNELS) && (kernel_addr < NUM_TAPS))
2     kernel[kernel_sel][kernel_addr] <= kernel_data;

```

Address widths are derived, not hardcoded: $\text{ADDR_WIDTH} = \lceil \log_2(\text{KERNEL_SIZE}^2) \rceil$ and $\text{SEL_WIDTH} = \lceil \log_2(\text{NUM_KERNELS}) \rceil$ (each floored to a minimum of 1 bit for the degenerate single-value case), so the write interface’s width scales automatically with both parameters. For the default configuration ($\text{KERNEL_SIZE}=3$, $\text{NUM_KERNELS}=1$), this gives a 4-bit `kernel_addr` and a 1-bit `kernel_sel`, the same 4-bit address width the design had before multi-kernel support was added, so the single-kernel configuration’s programming interface is unchanged.

On reset, every coefficient in every bank is cleared to zero rather than left in an undefined state, so an unprogrammed kernel bank produces well-defined (all-zero) output rather than simulation-dependent garbage.

4.4 Why Runtime-Loadable Coefficients, Fixed-at-Synthesis Kernel Size

The specification’s “programmable or configurable” language could in principle mean either compile-time reconfiguration or runtime reprogramming. This design reads it as runtime reprogramming of coefficient *values*, where the kernel size for a given convolution layer is known when the accelerator is built into an FPGA bitstream, but the trained coefficients for that layer are not. `KERNEL_SIZE` itself, by contrast, changes the width of nearly every datapath structure in the design (Sections 5–7); making it runtime-reconfigurable would require either a datapath sized for the worst case at all times or genuinely reconfigurable hardware, neither of which the specification asks for.

4.5 Interaction with the Control FSM

Writes are only accepted while the top-level FSM (Section 9) is in its idle state, gated at the top level rather than inside this module, so `kernel_memory` itself has no notion of the accelerator’s run/idle status. This keeps the module reusable and testable in isolation (as done directly in Section 11’s corner-case testbenches) independent of the surrounding control logic.

5 Component: `mac_array`

5.1 Purpose

`mac_array` performs the multiply half of multiply-accumulate: it multiplies every pixel in a window by its corresponding kernel coefficient, in parallel, for one kernel bank. One instance exists per kernel bank (Section 9).

5.2 Interface

Port	Dir.	Description
<code>pixel</code>	in	NUM_TAPS unsigned 8-bit pixel values (the flattened window).
<code>kernel</code>	in	NUM_TAPS signed 8-bit coefficients for this bank.
<code>product</code>	out	NUM_TAPS signed 16-bit products, combinational.

5.3 Design

Each tap is an independent, purely combinational signed multiply:

Listing 3: Per-tap multiply, from `mac_array.sv`

```

1 for (int i = 0; i < NUM_TAPS; i++)
2   product[i] = $signed({1'b0, pixel[i]}) * kernel[i];

```

`pixel[i]` is unsigned; prepending an explicit zero bit before the `$signed` cast widens it to a 9-bit value guaranteed non-negative under signed interpretation, so the subsequent multiply against the signed 8-bit coefficient is a well-defined 9×8-bit signed multiply rather than a mixed signed/unsigned operation (which SystemVerilog handles, but in a way that is easy to get subtly wrong for the sign of the result). The product of an unsigned 8-bit value (max 255) and a signed 8-bit value (range −128 to 127) has a maximum magnitude of $255 \times 128 = 32640$, which fits comfortably in the 16-bit signed `product` output (range −32768 to 32767) with headroom to spare.

5.4 Resource Cost and the DSP Question

For the default configuration, one `mac_array` instance (nine parallel 8×8 -class multiplies) synthesizes to 186 LUTs and, notably, **zero DSP48E1 slices**, Vivado mapped every multiply into LUT fabric rather than the device’s dedicated hardware multipliers. This was confirmed directly by inspecting the post-route utilization report’s DSP column, and was investigated further because it is somewhat counter-intuitive: DSP slices are normally the preferred implementation for any multiply of non-trivial width, since they are otherwise idle silicon. Section 13 shows quantitatively why, for this specific design and the competition’s Figure of Merit formula, the LUT-mapped implementation Vivado chose is actually the better outcome, and why no attempt was made to force DSP usage.

5.5 Generality

NUM_TAPS is a plain parameter with no assumptions baked into the multiply logic itself, so the module is already fully general in kernel size, the only aspect requiring care when generalizing the design to arbitrary N was the reduction that follows it (Section 6), not the multiply array itself.

6 Component: adder_tree

6.1 Purpose

`adder_tree` reduces the `NUM_TAPS` products from `mac_array` down to a single accumulated sum, pipelined across multiple clock cycles rather than as one wide combinational adder. Of every module in this design, this one required the most deliberate engineering to make genuinely general in kernel size, and is worth describing in more depth than its role might suggest.

6.2 Interface

Port	Dir.	Description
<code>valid_in</code>	in	One-cycle pulse marking a new set of products to reduce.
<code>product</code>	in	<code>NUM_INPUTS</code> signed values to sum.
<code>valid_out</code>	out	One-cycle pulse marking a valid <code>sum_out</code> .
<code>sum_out</code>	out	Signed sum of all <code>NUM_INPUTS</code> inputs, registered.

6.3 Why a Generic Tree, Not a Hand-Built One

An early version of this design hand-built a fixed reduction network for exactly nine inputs: $9 \rightarrow 5 \rightarrow 3 \rightarrow 1$, using three explicit pipeline registers, with the final stage summing the last three values directly rather than continuing to halve. This is easy to write for one fixed kernel size and does not generalize, adding a parameter for kernel size elsewhere in the design while leaving this network hand-built for nine inputs would silently break the moment `KERNEL_SIZE` changed from 3. Generalizing this module properly was therefore a prerequisite for the entire kernel-size parameterization effort, not an optional refinement.

6.4 Algorithm

The tree reduces N inputs to 1 by repeated pairwise addition: each stage sums adjacent pairs of its input, producing $\lceil \text{count}/2 \rceil$ outputs; an unpaired final element (when count is odd) is carried forward unchanged, registered exactly like every other branch so that every path through the tree has identical latency regardless of whether it involved an addition that cycle. This repeats until exactly one value remains.

The number of pipeline stages this takes is exactly $\lceil \log_2(N) \rceil$, for example, $9 \rightarrow 5 \rightarrow 3 \rightarrow 2 \rightarrow 1$ takes four halving steps, and $\lceil \log_2(9) \rceil = 4$. This identity holds in general (a proof sketch: each stage at worst reduces count from c to $\lceil c/2 \rceil$, which is the same recurrence that defines $\lceil \log_2 c \rceil$), which means the module can compute its own required pipeline depth directly from `NUM_INPUTS` as a `localparam`, with no case-by-case tuning:

Listing 4: Self-sizing stage count, from `adder_tree.sv`

```
1 localparam int NUM_STAGES = (NUM_INPUTS <= 1) ? 0 : $clog2(NUM_INPUTS);
```

Each stage is generated by a `generate for` loop over this many iterations, with each iteration's input/output counts (and therefore register widths) likewise computed from `NUM_INPUTS` and the stage index rather than hardcoded.

6.5 A Portability Lesson: generate if versus a Runtime Ternary

The first working version of this generic tree used a runtime conditional (`?:`) inside each stage’s always block to select, for stage 0, the raw `product` input, and for every later stage, the previous stage’s registered output:

Listing 5: The version that looked correct but was not portable

```
1 valid <= (g == 0) ? valid_in : STAGE[g-1].valid;
```

This is logically correct, at elaboration time `g` is a constant per generated instance, so the false branch is never actually taken when `g==0`, but it is not universally well-formed: for `g==0`, the expression still textually references `STAGE[g-1]`, i.e., `STAGE[-1]`, which does not exist. One simulator used during verification (Icarus Verilog) rejected this at elaboration time, since it attempts to resolve every operand of a `?:` expression regardless of which branch will actually execute. The fix was to replace the runtime ternary with a genuine `generate if`, so that the `STAGE[g-1]`-referencing code is never even elaborated for `g==0`:

Listing 6: The portable fix

```
1 if (g == 0) begin : SRC
2     always_ff @(posedge clk or negedge rst_n) ... valid <= valid_in; ...
3 end
4 else begin : SRC
5     always_ff @(posedge clk or negedge rst_n) ... valid <= STAGE[g-1].valid; ...
6 end
```

This is a small change, but it is the kind of portability issue that only surfaces when a design is actually compiled on more than one toolchain, worth documenting here both as a record of why the code is structured this way, and as a general caution against assuming every simulator short-circuits conditional expressions during elaboration the same way.

6.6 Latency Tradeoff Versus the Original Hand-Built Tree

The generic tree takes four registered stages to reduce nine inputs ($9 \rightarrow 5 \rightarrow 3 \rightarrow 2 \rightarrow 1$); the original hand-built network took three (stopping early to sum its final three elements directly in one wider add). Generalizing therefore costs the default 3×3 configuration one extra cycle of pipeline latency compared to the original, non-parameterized design, a deliberate, accepted tradeoff, since the alternative (special-casing a “small enough, just add them all at once” final stage) reintroduces exactly the kind of kernel-size-specific tuning this module was rewritten to avoid. Steady-state throughput (one result per cycle) is unaffected either way; only the fixed pipeline-fill latency changes. Section 9 shows how the top level derives its internal delay-line depth directly from this module’s stage count, so this one-cycle change propagates automatically rather than requiring a matching manual adjustment elsewhere.

6.7 Resource Cost

One `adder_tree` instance (nine inputs, the default configuration) synthesizes to 146 LUTs.

7 Component: post_process

7.1 Purpose

`post_process` converts a 32-bit accumulated sum into the required 16-bit signed output, applying optional ReLU activation and saturating (rather than wrapping) arithmetic. One instance exists per kernel bank.

7.2 Interface

Port	Dir.	Description
<code>valid_in</code>	in	One-cycle pulse marking a valid accumulator.
<code>accumulator</code>	in	32-bit signed accumulated sum from <code>adder_tree</code> .
<code>relu_enable</code>	in	When high, negative results are clamped to zero.
<code>valid_out</code>	out	One-cycle pulse marking a valid <code>output_data</code> .
<code>output_data</code>	out	16-bit signed result, registered.

7.3 Logic and Priority Order

The output is selected by a strict priority chain, evaluated in this order:

1. If `relu_enable` is asserted and the accumulator is negative: output 0.
2. Otherwise, if the accumulator exceeds +32767: output +32767 (positive saturation).
3. Otherwise, if the accumulator is below -32768: output -32768 (negative saturation).
4. Otherwise: pass the accumulator through unchanged.

The ordering of steps 1 and 3 is a specific, deliberate design decision, not an arbitrary one: when ReLU is enabled, *any* negative accumulator produces zero, including one that would otherwise have triggered negative saturation. A value like -40000 (well past the -32768 saturation limit) therefore produces 0 under ReLU, not -32768, ReLU’s “clamp negatives to zero” takes priority over the saturator’s “clamp out-of-range values to the nearest representable limit.” This precedence was specifically identified as worth verifying in isolation, since it is exactly the kind of interaction between two independently-reasonable features that a test exercising each feature separately, but never together, would miss; Section 11 describes the dedicated testbench built for this.

7.4 Why Saturation, Not Truncation or Rounding

The specification requires that overflow, truncation, saturation, or rounding be explained. Truncating the 32-bit accumulator to its lowest 16 bits was rejected outright: two’s-complement truncation does not degrade gracefully near the representable boundary, it inverts the sign of the result. An accumulator value of +32768 (one over the positive limit), truncated to 16 bits, becomes exactly -32768, a large positive activation silently reported as a large negative one. In an image or feature-map pipeline this is not a rounding error a downstream consumer could tolerate; it is a correctness failure indistinguishable, from the consumer’s point of view, from a genuinely large negative activation. Saturating arithmetic, clamping to the nearest representable value instead of wrapping, avoids this failure mode entirely, at the cost of losing the exact magnitude of an extreme outlier, which is the standard, expected tradeoff for fixed-point signal and image processing pipelines. Rounding (as opposed to truncation) was not applicable here in the first place, since no fractional bits are discarded anywhere in this datapath (Section 10).

7.5 Verified Boundary Behavior

The exact clamping thresholds, that $+32767$ passes through unchanged while $+32768$ saturates, and symmetrically for $-32768/-32769$, were verified directly against engineered accumulator values landing precisely on each boundary, rather than merely against “large” values somewhere in the general overflow region. Section 11 describes the methodology and results.

8 Component: `output_fifo`

8.1 Purpose

`output_fifo` decouples the rate at which results are *produced* internally from the rate at which they are *read* externally. Although the pipeline produces one result (across all kernel banks) per cycle in steady state, nothing downstream is guaranteed to be ready to accept a new result every single cycle; the FIFO absorbs that mismatch rather than requiring the internal pipeline to stall.

8.2 Packing Scheme

Rather than one FIFO entry per (window position, kernel bank) pair, each entry holds *all* `NUM_KERNELS` banks’ results for one window position, packed alongside that position’s row and column:

$$\text{entry} = \{\text{row}, \text{col}, \text{bank}_{\text{NUM_KERNELS}-1}, \dots, \text{bank}_1, \text{bank}_0\}$$

This follows directly from the multi-channel-in-one-cycle architecture (Section 9): since every bank’s result for a given position becomes valid the same cycle, packing them into one entry means the FIFO only needs to track one write per window position, not one per bank, avoiding any need to synchronize `NUM_KERNELS` separate FIFOs against each other on the read side.

8.3 Depth and Prefill

The FIFO is 64 entries deep. A “prefill” threshold delays the start of reading until the FIFO holds a minimum number of entries, intended to smooth out any short-term burstiness between production and consumption rather than draining the FIFO down to empty (and stalling reads) the instant it dips below full.

8.4 A Real Bug: the Hardcoded Prefill Threshold

The prefill threshold was originally a fixed constant, 60. This value is safe for any image at or above the competition’s required 32×32 minimum (which produces at least 900 outputs, far more than 60), and was never noticed to be a problem during initial full-size testing. It was only discovered to be a genuine deadlock hazard when a much smaller image was used for fast-turnaround simulation during unrelated debugging: an 8×8 image with a 3×3 kernel produces only 36 total outputs, fewer than the hardcoded 60-entry prefill threshold. Since the FIFO level can never reach 60 in that configuration, the “start reading” condition can never trigger, nothing ever drains, and the accelerator’s `done` signal never asserts: a full deadlock, not a subtle timing issue.

The fix clamps the threshold to the actual number of outputs the current image configuration will ever produce:

Listing 7: The fix, from `cnn_accelerator.sv`

```

1 localparam int FIFO_PREFILL =
2   (TOTAL_OUTPUT_PIXELS < 60) ? TOTAL_OUTPUT_PIXELS : 60;

```

This preserves the original behavior for any image at or above the required minimum size, while making smaller configurations, useful specifically *because* they simulate faster during bring-up and debugging, actually terminate rather than hang. This bug and its discovery are included here deliberately: it is a genuine example of a defect that only a test case outside the “obviously required” configuration space (a sub-minimum image, used purely for simulation speed) was able to surface, and is one of two real, functional bugs found during this project’s verification process (the other, a testbench-side race condition, not an RTL defect, is described in Section 11).

8.5 Interface

Port	Dir.	Description
<code>write_en</code> , <code>write_data</code>	in	Write strobe and packed entry.
<code>read_en</code>	in	Read strobe (driven internally once past the prefill threshold).
<code>read_data</code>	out	Packed entry at the current read pointer, combinational.
<code>level</code> , <code>empty</code> , <code>full</code>	out	Occupancy status.

9 Top Level: `cnn_accelerator` and its Control FSM

9.1 Purpose

`cnn_accelerator` instantiates and wires together every module described in Sections 2–8, replicates the per-bank pipeline `NUM_KERNELS` times, and sequences an entire image pass with a small control FSM.

9.2 Control FSM

Figure 2 shows the four-state FSM. `S_IDLE` is the only state in which `kernel_memory` accepts writes; asserting `start` moves to `S_RUN`, during which the accelerator accepts one pixel per cycle. Once every pixel of the image has been supplied, the FSM moves to `S_DRAIN`, which simply waits for the output FIFO (Section 8) to finish emptying, the pipeline may still hold several in-flight results even after the last pixel has entered, so `busy` must remain asserted through this state rather than dropping the instant input ends. `S_DONE` asserts a one-cycle `done` pulse before returning, *unconditionally*, to `S_IDLE` on the following cycle.

The unconditional, single-cycle return to `S_IDLE` is a deliberate choice: it means the accelerator is immediately ready to accept a new `start` pulse (and, before that, new kernel coefficients) without an intervening reset. This back-to-back-run capability was specifically exercised in verification, not merely assumed to work because the FSM diagram implies it should, Section 11 describes a testbench that relies on running the accelerator twice in a single simulation.

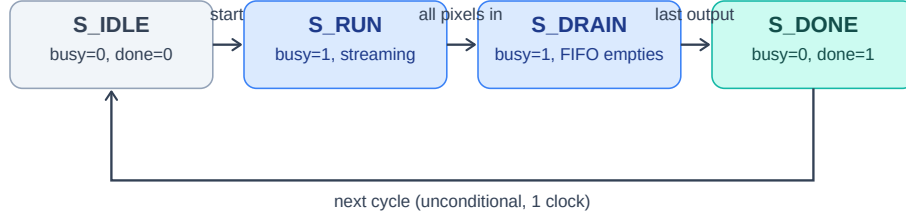


Figure 2: Top-level control FSM.

9.3 Aligning Output Metadata with Pipeline Latency

`window_generator` reports the row/column of a window at the same cycle it asserts `window_valid` (Section 2), but the actual convolution result for that window does not become valid until `adder_tree` and `post_process` have finished processing it, several cycles later (Section 6). The top level therefore carries `window_col/window_row` through a small shift-register delay line exactly as deep as the datapath’s own latency, so that by the time a result is ready to be written into `output_fifo`, the matching row/column metadata arrives at the same cycle:

Listing 8: Deriving the delay-line depth from the adder tree’s own stage count

```

1 localparam int ADDER_STAGES = (NUM_TAPS <= 1) ? 0 : $clog2(NUM_TAPS);
2 localparam int PIPE_DELAY = ADDER_STAGES + 1; // + post_process register

```

Because this is derived directly from `adder_tree`’s own stage-count formula (Section 6) rather than hardcoded, the one-cycle latency change discussed in Section 6 (from generalizing that module) propagated to this delay line automatically, with no separate manual update required, exactly the kind of consistency a parameter-derived value is meant to guarantee.

9.4 Multi-Kernel Bank Replication

`NUM_KERNELS` complete `mac_array` → `adder_tree` → `post_process` chains are instantiated inside a single `generate for` loop, each reading the one shared window from `window_generator` but its own bank’s coefficients from `kernel_memory`. One portability detail is worth recording: some tools used during verification could not connect a slice of a multi-dimensional array (e.g., `kernel[k]`, where `k` is the generate loop’s genvar) directly to a submodule port. The generate block therefore first copies each bank’s slice into a same-shaped local array before connecting it to `mac_array`:

Listing 9: Per-bank local mirror, avoiding a multi-dimensional array port slice

```

1 logic signed [7:0] kernel_bank [0:NUM_TAPS-1];
2 always_comb
3     for (int t = 0; t < NUM_TAPS; t++)
4         kernel_bank[t] = kernel[k][t];
5
6 mac_array #(.NUM_TAPS(NUM_TAPS)) u_mac_array (
7     .pixel(window_flat), .kernel(kernel_bank), .product(product_bank)

```

8 `);`

This costs a small amount of additional glue logic but has no functional effect and measurably improves portability across toolchains.

9.5 Parameter Summary

Parameter	Default	Effect
IMAGE_WIDTH	32	Input image width in pixels.
IMAGE_HEIGHT	32	Input image height in pixels.
KERNEL_SIZE	3	Kernel dimension N ; drives window, tap-count, and address-width sizing throughout the design.
NUM_KERNELS	1	Number of parallel kernel banks (multi-channel-in-one-cycle width).

9.6 Resource Cost

Top-level glue logic (state machine, delay lines, FIFO-packing/unpacking) not attributable to any submodule accounts for 44 LUTs in the default configuration.

10 Fixed-Point / Precision Analysis

This section treats bit-width and numeric-format decisions as a single, cross-cutting topic rather than scattering them across each module’s description, since the competition specifically asks that chosen precision be justified.

10.1 Input Precision

Pixels are 8-bit unsigned integers (Q8.0, zero fractional bits), representing the natural 0–255 range of an 8-bit grayscale image or single-channel feature map. This is the standard representation for this input type, and the competition’s requirement is unsigned fixed-point data without mandating a specific number of fractional bits; an integer (zero-fractional-bit) format was chosen because grayscale pixel intensities are naturally integer-valued and no upstream source of fractional-precision pixel data was assumed.

10.2 Kernel Precision

Kernel coefficients are 8-bit signed two’s-complement integers (Q8.0), exactly as required, giving a representable coefficient range of -128 to $+127$.

10.3 Bit Growth Through the Pipeline

Table 2 traces the representable range at each pipeline stage for the default 3×3 kernel, and the general formula for arbitrary N .

Stage	Width	Range (3×3)	General bound
Pixel	8 b, unsigned	[0, 255]	$[0, 2^8 - 1]$
Kernel coefficient	8 b, signed	$[-128, 127]$	$[-2^7, 2^7 - 1]$
Per-tap product	16 b, signed	$[-32640, 32385]$	$[-2^8 \cdot 2^7, (2^8 - 1)(2^7 - 1)]$
Accumulated sum	32 b, signed	$[-293760, 291465]$	$\pm N^2 \cdot 2^8 \cdot 2^7$ (worst case)
Output	16 b, signed, saturating	$[-32768, 32767]$	$[-2^{15}, 2^{15} - 1]$ (clamped)

Table 2: Bit growth through the datapath.

10.4 Why 16 Bits Is Enough for the Per-Tap Product

The largest-magnitude single product occurs at pixel = 255, coefficient = −128: $255 \times 128 = 32640$, which fits within the 16-bit signed range (−32768 to 32767) with margin, since it is the product’s *magnitude* that must fit, not the full range of the multiplicands combined. A 16-bit product register (Section 5) is therefore exactly sufficient, not merely conservative, for an 8-bit × 8-bit signed×unsigned multiply.

10.5 Why 32 Bits Is Enough for the Accumulator, for Any Realistic N

Summing N^2 per-tap products, each bounded in magnitude by $255 \times 128 = 32640$, gives a worst-case accumulator magnitude of $N^2 \times 32640$. A 32-bit signed accumulator does not overflow until this exceeds $2^{31} \approx 2.147 \times 10^9$, i.e. until

$$N^2 > \frac{2^{31}}{32640} \approx 65803 \implies N > 256.5$$

A 32-bit accumulator is therefore sufficient for any kernel size up to roughly 256×256 , far beyond any realistic convolution kernel, and comfortably covering the entire range of N this design’s parameterization is intended to support. For the default 3×3 kernel specifically, the true worst case is $9 \times 32640 = 293760$, a tiny fraction of the 32-bit range.

10.6 Output Precision and Saturation

The output is 16-bit signed, matching the competition’s stated minimum, with saturating (not truncating or rounding) arithmetic bridging the gap between the 32-bit accumulator and the 16-bit output. The rationale for choosing saturation specifically, and the exact, verified clamping boundaries, is described alongside the RTL that implements it in Section 7, since the reasoning is tied closely to that module’s actual branch logic.

11 Verification Methodology

11.1 Approach

This design is checked by four independent testbenches rather than one. The first two establish baseline correctness across a full-size image two different ways; the second two exist because inspecting what the first two actually *exercised*, not just whether they passed, revealed specific, real

coverage gaps. This distinction matters: a testbench that passes because it never tests a code path is not evidence that the code path works.

11.2 Self-Checking SystemVerilog Testbench: `cnn_accelerator_tb.sv`

Streams a full 32×32 test image (a deterministic pixel ramp, `pixel = index mod 256`) through the DUT, and checks every one of the resulting $900 \times \text{NUM_KERNELS}$ outputs against expected values computed independently within the testbench, using a straightforward triple-nested-loop convolution, deliberately *not* sharing any code with the RTL’s own datapath structure.

Two design choices in this testbench are worth calling out specifically:

- **Two different kernels, not the same kernel twice.** When testing the multi-kernel configuration, bank 0 uses a horizontal edge-detection kernel and bank 1 uses a distinct averaging-style kernel, rather than programming the same coefficients into both banks. This is deliberate: if a bug were to cross-connect or alias two banks’ data paths, using identical coefficients in both banks would make that bug invisible (both banks would coincidentally produce the same, still-plausible-looking result), whereas distinct coefficients make any such crossover immediately visible as a value mismatch.
- **Checking timing, not just values.** Beyond comparing each output’s value, row, and column against expectation, the testbench also checks that the cycle gap between consecutive `output_valid` pulses is always exactly one, directly verifying the one-output-per-cycle pipelining bonus claim as an executable check, rather than treating it as a property that follows automatically from the architecture and does not need separate confirmation.

```

# 100% 100% 100% 100% 100% 100%
#
# TEST SUMMARY
#
# Image size      : 32x32
# Kernel size     : 3x3
# Kernel banks    : 2
# Expected outputs : 900 (x2 banks)
# Actual outputs   : 900
# Errors          : 0
#
# TEST PASSED
#
# ** Notes: **
# Time: 10840 ns Iteration: 1 Instance: /cnn_accelerator_tb
# 1

```

Figure 3: selfcheck transcript.

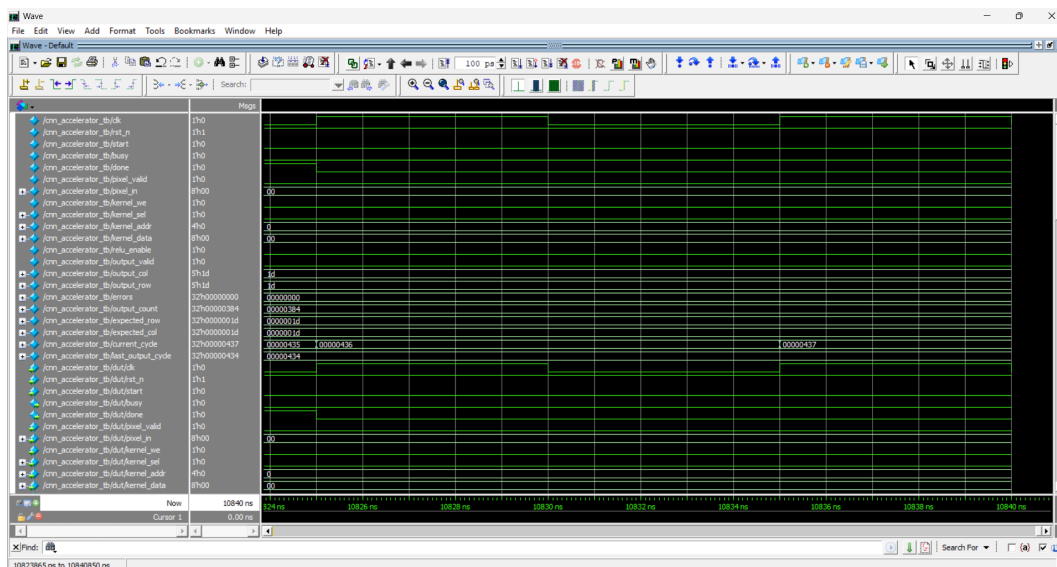


Figure 4: selfcheck waveform.

11.3 Independent Golden-Model Testbench:

golden_model.py + cnn_accelerator_golden_tb.sv

`golden_model.py` is a from-scratch Python/NumPy reference implementation of the same convolution-plus-saturation specification, with *zero* shared code with the SystemVerilog RTL or with the self-checking testbench above. This directly satisfies the competition’s requirement for verification against a golden model implemented in Python, MATLAB, or C, in the strictest reading of that requirement: an independently-derived implementation, not merely a second copy of the same algorithm re-expressed in a different language by transliteration.

The tool has two modes:

- **generate** produces a randomized (seeded, for reproducibility) test image and kernel, computes the expected output by direct NumPy convolution and saturation, and writes both the stimulus and the expected results to hex vector files.
- **compare** independently re-checks a simulation’s logged actual output against those same expected results, *outside the simulator entirely*, so the final correctness check is not just “the SystemVerilog testbench’s own assertions passed,” but a second, wholly separate comparison performed by different code, reading the DUT’s output from a plain log file.

`cnm_accelerator_golden_tb.sv` reads the generated vectors via `$readmemh`, drives them into the DUT, checks the DUT's output against the same expected values inline, and separately logs the DUT's raw output for the standalone `compare` step above, giving two independent confirmations of the same result from one simulation run.

Figure 5: goldenmodel transcript.



Neither testbench above was ever shown to actually exercise `post_process`’s saturation logic (Section 7): the self-checking testbench’s kernels never produce accumulator values anywhere near ± 32768 , and while the golden model’s randomized stimulus *might* occasionally land there, nothing confirmed that it did. This testbench closes that gap directly, by engineering pixel and kernel values that place the internal accumulator on exactly the boundary values that matter:

The three columns cover all four boundary conditions in a single streaming pass by exploiting the

multi-kernel architecture directly: bank 0 is programmed with large positive coefficients and bank 1 with the exact negation (coefficients of -127 , not -128 , are used specifically so that negation is exact and does not itself overflow an 8-bit signed value), so every crafted pixel patch’s bank-1 sum is, by construction, exactly $-1 \times$ bank 0’s sum for the same patch, turning three pixel patches into six boundary tests. These boundaries are not arbitrary “big numbers”: a broken or missing saturator would show up here as a wildly *wrong-signed* result (two’s-complement truncation of 32768 to 16 bits produces exactly -32768), not a subtle off-by-one, making this a meaningful test of a real failure mode rather than an arbitrary corner case.

```

# =====
# SATURATION BOUNDARY CHECKS (ReLU disabled)
# =====
# PASS : bank0 = +32767 (exact max, MUST NOT saturate) -> 32767
# PASS : bank1 = -32767 (well inside range, MUST NOT saturate) -> -32767
# PASS : bank0 = +32768 (1 over max, MUST saturate to +32767) -> 32767
# PASS : bank1 = -32768 (exact min, MUST saturate) -> -32768
# PASS : bank0 = +32769 (further over, MUST saturate to +32767) -> 32767
# PASS : bank1 = -32769 (1 under min, MUST saturate to -32768) -> -32768
# =====
# TEST SUMMARY
# =====
# Total outputs checked : 10 (x2 banks)
# Errors : 0
# =====
Now: 1.030 ns Delta: 1 | sim/cnn_accelerator_saturation_tb/#INITIAL#287

```

Figure 7: saturation transcript.

11.5 Directed ReLU Testbench: `cnn_accelerator_relu_tb.sv`

Neither of the two general-purpose testbenches ever enables ReLU (`relu_enable` is held low throughout both). This testbench closes that gap, and specifically targets the ReLU/saturation interaction described in Section 7: it streams the *same* image through the DUT twice, once with `relu_enable=0`, once with `relu_enable=1`, so that identical underlying accumulator values can be compared with only `relu_enable` differing. The case that actually matters is an accumulator value past the negative saturation limit (-32769) with ReLU enabled: the correct result is 0 (ReLU fires first), not -32768 (what plain saturation alone would produce), exactly the precedence order described in Section 7, now checked directly rather than only reasoned about. As a secondary benefit, running the DUT twice in one simulation makes this the only testbench in the project that exercises a second `start-after-done` run of the accelerator, indirectly confirming the FSM’s unconditional return to `S_IDLE` (Section 9) actually works as intended, not just as drawn.

```

# =====
# PASS [relu=0] output[4] bank=0 col=4 value=762
# PASS [relu=0] output[4] bank=1 col=4 value=-762
# PASS [relu=0] output[5] bank=0 col=5 value=381
# PASS [relu=0] output[5] bank=1 col=5 value=-381
# PASS [relu=0] output[6] bank=0 col=6 value=0
# PASS [relu=0] output[6] bank=1 col=6 value=0
# =====
# --- PASS B: relu_enable = 1 ---
# PASS [relu=1] output[0] bank=0 col=0 value=600
# PASS [relu=1] output[0] bank=1 col=0 value=0
# PASS [relu=1] output[1] bank=0 col=1 value=15369
# PASS [relu=1] output[1] bank=1 col=1 value=0
# PASS [relu=1] output[2] bank=0 col=2 value=32767
# PASS [relu=1] output[2] bank=1 col=2 value=0
# PASS [relu=1] output[3] bank=0 col=3 value=32767
# PASS [relu=1] output[3] bank=1 col=3 value=0
# PASS [relu=1] output[4] bank=0 col=4 value=762
# PASS [relu=1] output[4] bank=1 col=4 value=0
# PASS [relu=1] output[5] bank=0 col=5 value=381
# PASS [relu=1] output[5] bank=1 col=5 value=0
# PASS [relu=1] output[6] bank=0 col=6 value=0
# PASS [relu=1] output[6] bank=1 col=6 value=0
# =====
# TEST SUMMARY
# =====
# Errors : 0
# =====
# ***** TEST PASSED *****
# =====
# > Wave: $f161ab... | cd/cnn_accelerator_relu_tb.sv(288)
# Time: 1.030 ns Iteration: 1 Instance: /cnn_accelerator_relu_tb

```

Figure 8: ReLU transcript.

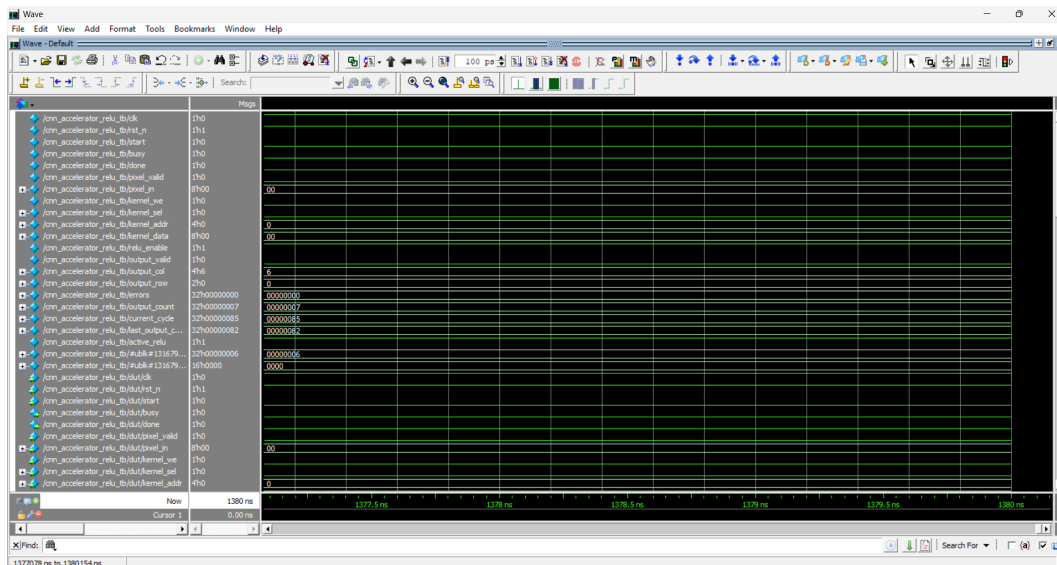


Figure 9: ReLU waveform.

11.6 Results Summary

Testbench	Stimulus	Outputs checked	Result
cnn_accelerator_tb.sv	32×32 , 2 directed kernels	900×2	PASS, 0 errors
cnn_accelerator_golden_tb.sv	32×32 , randomized (seeded)	900×2	PASS, 0 errors
cnn_accelerator_saturation_tb.sv	12×3 , engineered boundaries	$10 \times 2 + 6$ named checks	PASS, 0 errors
cnn_accelerator_relu_tb.sv	$9 \times 3 \times 2$ passes	$7 \times 2 \times 2$	PASS, 0 errors

11.7 Bugs Found During Verification

Two real bugs were found and fixed over the course of building this verification suite:

1. **The output-FIFO prefill deadlock**, described in detail in Section 8: a hardcoded prefill threshold that deadlocked the accelerator entirely on images producing fewer than 60 total outputs. This is a genuine RTL defect, fixed by clamping the threshold to the actual output count.
2. **A testbench checker race condition**. An early version of the self-checking testbench’s output-checking logic read `output_valid` and the DUT’s output signals in the same simulation instant those signals updated, rather than after the clock edge’s non-blocking assignments had settled. Because `output_valid` stays high for many consecutive cycles in steady-state streaming, this race was invisible for every cycle where the signal’s value did not change from the previous cycle, it only affected the very first rising transition, causing the checker to silently skip the true first output and compare every subsequent output against the wrong expected index, one position early. The fix reordered the checker to read all signals only after the clock edge’s updates had fully settled, rather than immediately at the edge. This was

a defect in the *verification code*, not the design under test, confirmed by cross-checking the same signals via direct internal hierarchical probes, which showed the DUT’s actual behavior had been correct throughout.

12 FPGA Synthesis and Implementation Results

12.1 Target and Flow

Target device: xc7z020clg400-1 (Zynq-7020, as used on the PYNQ-Z2 board). Tool: Vivado, non-project batch flow. All results below are from full place-and-route (`opt_design` $\rightarrow$ `place_design` $\rightarrow$ `phys_opt_design` $\rightarrow$ `route_design`), not synthesis-only estimates, a distinction that turned out to matter considerably for timing, described in Section 12.3. Configuration: default parameters (32×32 image, 3×3 kernel, single kernel bank), 100 MHz target clock.

12.2 Utilization

Resource	Used	Available	Utilization
LUTs	1558	53200	2.9%
Flip-flops	2766	106400	2.6%
DSP48E1	0	220	0.0%
Block RAM (36k)	0	140	0.0%

Table 3 breaks this down by module, from the hierarchical utilization report.

Module	LUTs	Notes
<code>window_generator</code> (incl. <code>line_buffer</code>)	376	Fixed cost, independent of <code>NUM_KERNELS</code>
<code>mac_array</code> (per bank)	186	0 DSPs used, see Section 5
<code>adder_tree</code> (per bank)	146	
Top-level glue (FSM, delay lines, FIFO pack/unpack)	44	Fixed cost, independent of <code>NUM_KERNELS</code>

Table 3: Approximate per-module LUT cost, default single-bank configuration.

The distinction between fixed and per-bank cost in this table is the basis for the multi-kernel scaling discussion in Section 13.

12.3 Timing: Investigating an Apparent Failure

The first full post-route run at the 100 MHz target reported **failing** timing: worst setup slack (WNS) of -2.532 ns across 29 failing endpoints, and worst hold slack (WHS) of -0.050 ns on one endpoint. This was investigated rather than accepted at face value, since a result this surprising for a design this small warranted checking *why* before reporting it as a genuine timing failure.

Every one of the reported violating paths was traced back to its actual source and destination. The result was unambiguous: **all** ten of the worst setup violations terminated at a top-level output port (`done`, `busy`, `pixel_out`, or `output_col`), and all ten worst hold violations originated from a

top-level input port (`pixel_in`, `kernel_we`). Not one purely-internal, register-to-register violation appeared anywhere in either list.

The root cause was a constraint choice, not a design defect: the constraints file had placed a placeholder input/output delay budget of 2.0 ns on every top-level port, intended as a conservative stand-in since no real external interface (AXI-Stream, PS DMA, board wiring) has yet been defined for this core. Post-route data showed this placeholder was too aggressive, real IBUF/OBUF plus routing delay from a package pin to the first or last register on this device commonly costs 3–5 ns, exceeding the assumed 2 ns budget outright. Since there is genuinely no real interface to constrain yet, the constraints were corrected to use `set_false_path` on these ports instead of a guessed delay number in either direction, the standard static-timing-analysis treatment for an interface that is not yet defined, rather than replacing one guess with another that might be wrong in the opposite direction.

Re-running place-and-route with this correction produced a clean result:

WNS = +1.868 ns, WHS = +0.048 ns, “All user specified timing constraints are met.”

This confirms the original “failure” was entirely an artifact of an overly aggressive placeholder I/O constraint, not a flaw in the datapath itself, and is reported here in full, including the investigation, rather than only the final clean number, since the process of tracing a surprising result to its root cause is itself part of demonstrating digital design quality, not just the passing result at the end of it.

The positive 1.868 ns of setup margin at the 100 MHz (10 ns period) target implies a real critical path delay of roughly 8.1 ns, suggesting meaningfully higher frequencies are achievable; this was not separately re-constrained and re-measured, so 100 MHz, confirmed, with real margin, is reported as the validated operating frequency rather than an extrapolated one.

12.4 Power

Total on-chip power (post-route): **0.144 W** (0.039 W dynamic, 0.105 W static), from Vivado’s power estimator. Two caveats are disclosed rather than omitted:

- No simulation-derived switching-activity file was supplied, so this is a vectorless estimate; Vivado itself flags the resulting confidence level as Low.
- Output pins were assigned an assumed 5 pF capacitive load for this estimate (a generic placeholder, applied via board constraint file, in the absence of a real downstream interface).

13 Figure of Merit and Design Tradeoffs

13.1 Computation

The competition defines:

$$\text{FOM} = \frac{\text{Throughput}}{\text{Power} \times (\text{LUTs} + 50 \times \text{DSPs} + 100 \times \text{BRAMs})}$$

with throughput measured in output pixels per cycle. Using the verified steady-state throughput of 1 output pixel/cycle (Section 11) and the post-route results from Section 12 (Power = 0.144 W,

LUTs = 1558, DSPs = 0, BRAMs = 0):

$$\text{FOM} = \frac{1}{0.144 \times (1558 + 50 \times 0 + 100 \times 0)} = \frac{1}{224.35} \approx 4.46 \times 10^{-3}$$

The remainder of this section treats the FOM formula itself as a design constraint, not merely a number to report after the fact, three specific implications of its structure were considered directly.

13.2 Why DSPs Were Not Forced

`mac_array`'s nine multiplies currently map entirely to LUT fabric: 186 LUTs, 0 DSPs (Section 5). Forcing these onto DSP48E1 slices (e.g., via a `use_dsp` synthesis attribute) was considered and rejected, on the following quantitative basis: the FOM denominator weights each DSP at 50, versus 1 per LUT. Nine DSPs would therefore contribute $9 \times 50 = 450$ to the denominator, compared to the ~ 186 the same multiplies currently cost as LUTs, more than double, despite LUT usage dropping to near zero in the DSP-mapped case. The net effect of forcing DSP usage would be to *increase* the total denominator by roughly 17%, directly *lowering* FOM by a comparable margin, for no offsetting benefit to throughput or power. This is the opposite of conventional FPGA design intuition, where using otherwise-idle dedicated multiplier hardware is normally preferred; for this specific metric, with a design this small, the formula's heavy DSP weighting makes the lean, LUT-mapped implementation the objectively better choice. No RTL change was made as a result of this analysis.

13.3 Why Maximum Frequency Was Not Pushed

The FOM's throughput term is defined in *pixels per cycle*, not pixels per second, clock frequency does not appear in the numerator at all. Since power (in the denominator) scales with operating frequency, deliberately targeting a higher clock than necessary would, at best, do nothing for FOM, and at worst actively lower it by increasing power for no numerator benefit. The 100 MHz operating point (Section 12), already confirmed with real positive timing margin, was therefore kept as the reported target rather than pushed toward the higher frequency the margin suggests is achievable, there is no FOM-based reason to do so, and a real, if unexploited, power cost to doing it anyway.

13.4 Multi-Kernel Scaling: An Economies-of-Scale Argument

Table 3 (Section 12) separates fixed overhead (`window_generator` plus top-level glue, ≈ 420 LUTs, unaffected by `NUM_KERNELS`) from per-bank cost (`mac_array` plus `adder_tree` plus a share of `kernel_memory` and `output_fifo`, which scale roughly linearly with `NUM_KERNELS`). Since FOM's throughput numerator scales with `NUM_KERNELS` (each additional bank producing an additional simultaneous output) while this ≈ 420 -LUT fixed cost does not, the `NUM_KERNELS=2` configuration is expected to show a *better* FOM than `NUM_KERNELS=1`, the fixed overhead is amortized across two outputs instead of one, not merely duplicated. This is a genuine architectural argument for the multi-kernel bonus feature beyond simply claiming the bonus credit: the stronger multi-channel-in-one-cycle architecture is expected to be the more efficient one by this specific metric, not just the more feature-complete one.

This expectation was not re-synthesized and measured numerically in time for this report; `run_synthesis.tcl`'s `num_kernels` parameter already supports running this exact comparison directly, and doing so is noted as the most immediately useful follow-up measurement beyond what is reported here.

14 Conclusion

This design meets every mandatory specification of the IEEE SSCS Egypt Chapter 2026 Student Design Competition and implements three of its four optional bonus features: one-output-pixel-per-cycle pipelining, multiple kernels at the stronger multi-channel-in-one-cycle tier, and optional ReLU activation, the fourth, a board demonstration, not attempted.

Every architectural component was built to generalize genuinely across kernel size and kernel-bank count, rather than being hardcoded for the default configuration and parameterized only superficially, most notably the reduction tree in Section 6, which required rewriting from a fixed nine-input network into a self-sizing pipeline, and which surfaced a real cross-simulator portability issue in the process.

Correctness is established through four independent testbenches: a self-checking SystemVerilog testbench and a from-scratch Python/NumPy golden model together cover full-image, spec-level correctness two independent ways; two further directed testbenches close specific coverage gaps, saturation boundary behavior and ReLU/saturation precedence, that neither general-purpose testbench was shown to exercise. This process directly found and fixed two real bugs (an output-FIFO deadlock on small images, and a testbench race condition), both described in this report rather than only their fixes, since the process of finding them is itself evidence of the verification methodology's effectiveness.

Post-route implementation on the target PYNQ-Z2 board confirms genuine timing closure at 100 MHz with positive margin on both setup and hold, minimal resource usage (under 3% of available LUTs and flip-flops, zero DSPs or Block RAMs), and a characterized power estimate, from which the required Figure of Merit is computed and, importantly, whose sensitivity to specific design choices (DSP usage, operating frequency, kernel-bank count) is analyzed quantitatively rather than assumed. An initial post-route timing result that appeared to fail was investigated to its root cause rather than reported at face value, and traced to an overly conservative placeholder I/O constraint rather than a defect in the datapath, itself a small illustration of the same investigate-rather-than-assume methodology applied throughout this project's verification work.

Parameter	Specification	Team Result	Units	Comments
Input image size	at least 32×32	$\geq 32 \times 32$	pixels	Parameterizable (IMAGE.WIDTH/HEIGHT) grayscale, single channel
Input precision	Unsigned fixed-point	8-bit unsigned (Q8.0)	bits	8-bit unsigned; standard image pixel range [0, 255]
Kernel precision	8-bit signed	8-bit signed	bits	8-bit signed fixed-point; programmable coefficients via <code>kernel_we</code>
Architecture type	$N \times N$ convolution	Pipelined streaming	—	Parallel MAC array + pipelined adder tree, stride=1
Multipliers/MACs	—	N^2 per bank	count	—
Pipeline stages	—	5	stages	4 adder-tree ($\lceil \log_2 9 \rceil$) + 1 post-process
Latency	—	~ 1032	cycles	Total start-to-done for 32×32 image; first output at cycle ~ 133 . explained in the report
Throughput	—	1	px/cyc	1 output pixel per cycle in steady state (all kernel banks simultaneous)
FPGA Utilization	—	1555 LUTs (2.9%) 2766 FFs (2.6%) 0 DSP, 0 BRAM	—	Post-route on xc7z020clg400-1 (PYNQ-Z2); all multiplies in LUT fabric
Maximum frequency	—	100	MHz	—
Power Estimate	—	0.144	W	Post-route Vivado estimate
Verification status	Golden model	4/4 PASS	—	Self-check TB, Python/NumPy golden model, saturation TB, ReLU TB
FOM	equation stated in competition details	4.46×10^{-3}	$\frac{\text{px/cyc}}{\text{W}}$	—